



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Solving Chess Endgames

A Reinforcement Learning Approach

Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

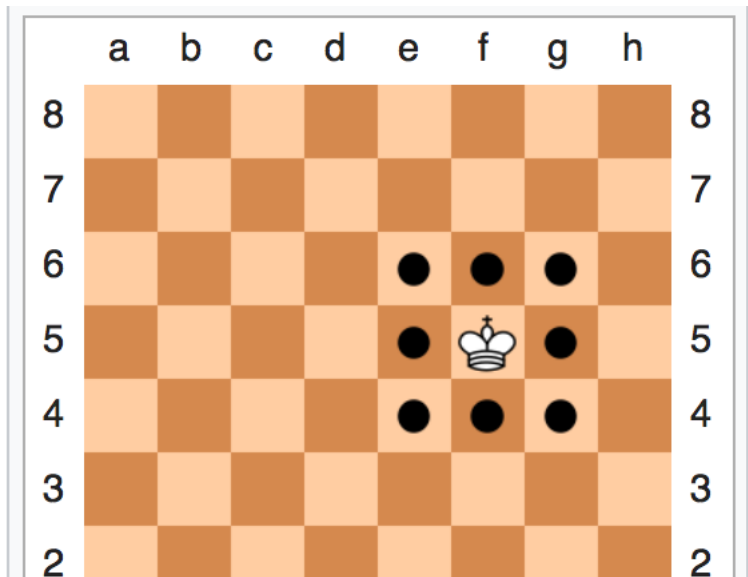
September 5, 2025

Chess Programming Background

Brief History of Chess Programming

- 1890 by the Spanish engineer Leonardo Torres y Quevedo KRK
- 1950 that Claude Shannon published the first article about programming a computer for playing chess.
- 1997 *Deep Blue*: the first computer program to beat the current world champion.
- Stockfish
- 2017 *Alphazero* defeated Stockfish
- from 2018, more models developed based on Alphazero.

Chess Rules



Our Rules

Endgame Phase

- * what is an Endgame

In parallel, Bellman in 1965 developed the idea of retrograde analysis for computing endgame tables. Using this idea, Thompson and Stiller in the nineties solved all chess endgames with up to five pieces.

In 2012 Makhnychev and Zakharov compiled the ****Lomonosov Endgame Tablebase****, which solved all endgame positions with up to seven pieces—some require over 500 moves without a capture. The 7-piece table consumes 140 terabytes; an 8-piece table would be 100 times larger.

Simple Endgame: (our) Problem Definition

Simple Endgames that we want to solve:

- * winning for white
- * kings and one (or more) heavy piece for white
- * the number of states is already huge

Assessment Metrics Overview

* DTM, DTZ etc * success, top1 etc

MDP Formulation

Markov Game vs MDP

- * States $\mathcal{S}$: all legal endgame positions, augmented with side-to-move (no turn number, no 50 moves rule)
- * Terminal State $\bar{\mathcal{S}}$: checkmate or draw (insufficient material), once reached the game ends.

* Actions: legal moves for the current player.

MDP: Transition

* Transition function: deterministic update given current state and chosen action, followed by the opponent's (black) deterministic reply

Instead of storing $\mathcal{P}$, chess engines implement functions that *define* $\mathcal{P}$ procedurally:

* $legal_{moves}(s)$ — generates valid actions in state s

* $apply_{move}(s, a)$ — returns the next state s'

* $is_{terminal}(s')$ — checks if game is over

MDP: Reward

- * Rewards (Chess): +1 for win, -1 for loss, 0 otherwise.
- * Rewards (simple endgames): -2 per ply, -1000 for draw
- * gamma issue

RL Algorithms

Results

Policy interpretability through human chess principles

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Works

- * zobrist hashing and invariance properties in chess endgames to reduce state space cardinality

Thank You!